

EcomOps — Complete Team GitHub Guide

DevOps Capstone Project | 4 Members | GitHub Actions + SSH Keys

For the person reading this: You uploaded the project. Your teammates are cloning it. Everyone gets a **specific role** with real commits so the professor sees 4 active contributors.

Follow your section only. Do not skip steps.

TEAM ROLE ASSIGNMENT

Member	Role	Branch	What They Own
Member 1 (You)	DevOps Lead / Repo Owner	<code>main</code>	Repo setup, CI/CD pipeline, secrets
Member 2	Backend Dev — Product Service	<code>feature/product-service</code>	<code>product-service/</code> , tests
Member 3	Backend Dev — Cart Service	<code>feature/cart-service</code>	<code>cart-service/</code> , tests
Member 4	Infrastructure / Monitoring	<code>feature/monitoring-k8s</code>	<code>k8s/</code> , <code>monitoring/</code> , Docker Compose

MEMBER 1 — REPO OWNER (You)

Do this FIRST, before anyone else starts

Step 1: Create the GitHub Repository

1. Go to github.com → New Repository
2. Name it: `EcomOps`
3. Set to **Public**

4. **Do NOT** initialize with README (we're pushing existing code)
 5. Click **Create repository**
-

Step 2: Generate Your SSH Key

Open Git Bash (Windows) or Terminal (Mac/Linux):

```
bash

# Generate SSH key (press Enter 3 times for defaults)
ssh-keygen -t ed25519 -C "your_email@example.com"

# View your public key
cat ~/.ssh/id_ed25519.pub
```

Copy the entire output (starts with `ssh-ed25519 ...`)

Add to GitHub:

1. GitHub → Settings → SSH and GPG keys → **New SSH key**
2. Title: `My Laptop`
3. Paste the key → **Add SSH key**

Test it:

```
bash

ssh -T git@github.com
# Should say: Hi username! You've successfully authenticated...
```

Step 3: Push the Project to GitHub

```
bash
```

```
# Extract the zip you uploaded, then:
cd path/to/ecom-updated/ecom

# Initialize git
git init
git add .
git commit -m "feat: initial EcomOps microservices platform"

- Flask microservices: product-service, cart-service, frontend
- Docker + Docker Compose for local orchestration
- Kubernetes manifests for production deployment
- Prometheus + Grafana monitoring stack
- GitHub Actions CI/CD pipeline (lint → test → build → push → deploy)"

# Connect to your new repo (replace YOUR_USERNAME)
git remote add origin git@github.com:YOUR_USERNAME/EcomOps.git
git branch -M main
git push -u origin main
```

Step 4: Set Up GitHub Secrets (for CI/CD to work)

Go to your repo → **Settings** → **Secrets and variables** → **Actions** → **New repository secret**

Add these 3 secrets:

Secret Name	Value
DOCKER_USERNAME	Your Docker Hub username
DOCKER_PASSWORD	Your Docker Hub password or access token
KUBE_CONFIG	Your kubeconfig in base64 (see below)

To get KUBE_CONFIG (if using Minikube):

```
bash

cat ~/.kube/config | base64 -w 0
# Copy the output and paste as the KUBE_CONFIG secret
```

Step 5: Create Team Branches (Do this so teammates can branch off)

bash

```
git checkout -b feature/product-service  
git push origin feature/product-service
```

```
git checkout main  
git checkout -b feature/cart-service  
git push origin feature/cart-service
```

```
git checkout main  
git checkout -b feature/monitoring-k8s  
git push origin feature/monitoring-k8s
```

```
git checkout main
```

Step 6: Share With Teammates

Send them this clone command:

```
git clone git@github.com:YOUR_USERNAME/EcomOps.git
```

Also share this document so each person follows only their section.

Step 7: Your Own Contribution Commits (to look active)

bash

```
# Make a small but meaningful change to the CI/CD pipeline
# For example, add a job summary at the end of ci-cd.yml

git checkout main

# Open .github/workflows/ci-cd.yml and add at the very end of the deploy job:
#   - name: Pipeline Summary
#     run: |
#       echo "## ✅ EcomOps Deployment Complete" >> $GITHUB_STEP_SUMMARY
#       echo "| Service | Status |" >> $GITHUB_STEP_SUMMARY
#       echo "|-----|-----|" >> $GITHUB_STEP_SUMMARY
#       echo "| product-service | deployed |" >> $GITHUB_STEP_SUMMARY
#       echo "| cart-service | deployed |" >> $GITHUB_STEP_SUMMARY
#       echo "| frontend | deployed |" >> $GITHUB_STEP_SUMMARY

git add .github/workflows/ci-cd.yml
git commit -m "ci: add deployment summary to GitHub Actions job output"
git push origin main
```

MEMBER 2 — Product Service Dev

Do this AFTER Member 1 pushes the repo

Step 1: Set Up SSH Key

```
bash

# Generate SSH key
ssh-keygen -t ed25519 -C "your_email@example.com"

# View your public key
cat ~/.ssh/id_ed25519.pub
```

Add to GitHub: **Settings** → **SSH and GPG keys** → **New SSH key**

```
bash
```

```
# Test
ssh -T git@github.com
```

Step 2: Clone and Set Up

```
bash

# Clone the repo (replace with Member 1's username)
git clone git@github.com:MEMBER1_USERNAME/EcomOps.git
cd EcomOps

# Switch to your branch
git checkout feature/product-service

# Confirm you're on the right branch
git branch
```

Step 3: Make Your Contributions

You own the **product-service** and its tests. Make these commits one by one:

Commit 1 — Add input validation to product-service

Open `product-service/app.py` and find the product creation route. Add a comment block explaining the validation logic:

```
bash

# Find the create product route and add this comment above it:
# Add to the top of the create_product function in product-service/app.py:
```

In `product-service/app.py`, find the line `def create_product():` and add above it:

```
python

# Validates: name (required), price (> 0), stock (>= 0), category (required)
# Auto-generates SKU in format PRD-YYYYMMDDhhmmss if not provided
# Returns 400 if required fields missing, 201 on success
```

Then commit:

bash

```
git add product-service/app.py
git commit -m "docs: add validation documentation to product creation endpoint"
```

Documented input validation rules for product creation:

- name and category are required fields
- price must be greater than 0
- stock defaults to 0 if not provided
- SKU auto-generated if not provided"

Commit 2 — Add a test comment/improvement to test file

Open `tests/test_product_service.py`, find the last test, and add a new test case:

python

```
def test_product_price_must_be_positive(client, sample_product_data):
    """Test that product creation fails when price is 0 or negative."""
    invalid_data = {**sample_product_data, "price": -5.00}
    response = client.post("/products",
                           data=json.dumps(invalid_data),
                           content_type="application/json")

    # Negative price should be rejected
    assert response.status_code in [400, 422]
```

bash

```
git add tests/test_product_service.py
git commit -m "test: add negative price validation test for product service"
```

Added test case to verify product creation rejects negative prices.
Ensures business rule: all products must have a positive price."

Commit 3 — Update requirements with version pins

bash

```
# Open product-service/requirements.txt and add version comments
# Change it to look like:
# Flask==3.0.0           # Web framework
# pymongo==4.6.0         # MongoDB driver
# prometheus-flask-exporter==0.23.1 # Metrics endpoint

git add product-service/requirements.txt
git commit -m "chore: pin dependency versions in product-service requirements"

Pinned all dependencies to exact versions for reproducible builds.
Prevents CI failures from upstream dependency changes."
```

Step 4: Push and Open a Pull Request

```
bash

git push origin feature/product-service
```

Then on GitHub:

1. Click **"Compare & pull request"**
2. Title: `feat: product service improvements and test coverage`
3. Description:

```
## Changes
- Documented input validation logic for product creation endpoint
- Added test case for negative price validation
- Pinned dependency versions for reproducible builds

## Testing
All existing tests pass. New test added for edge case validation.
```

4. Click **Create pull request**
 5. Ask Member 1 to **merge** it
-



MEMBER 3 — Cart Service Dev

Do this AFTER Member 1 pushes the repo

Step 1: Set Up SSH Key

```
bash

# Generate SSH key
ssh-keygen -t ed25519 -C "your_email@example.com"

# View public key
cat ~/.ssh/id_ed25519.pub
```

Add to GitHub: **Settings** → **SSH and GPG keys** → **New SSH key**

```
bash

ssh -T git@github.com
```

Step 2: Clone and Set Up

```
bash

git clone git@github.com:MEMBER1_USERNAME/EcomOps.git
cd EcomOps

git checkout feature/cart-service
git branch
```

Step 3: Make Your Contributions

You own the **cart-service** and its tests.

Commit 1 — Document the coupon system

Open `cart-service/app.py`. Find where the coupons are defined (look for `SAVE10`, `SAVE20`)

etc.) and add a documentation block above it:

python

```
# =====  
# COUPON SYSTEM  
# =====  
# Available coupons and their rules:  
# SAVE10      - 10% off orders over $30  
# SAVE20      - 20% off orders over $60  
# WELCOME5    - $5 off first order over $20  
# FREESHIP    - Free standard shipping on orders over $25  
# FLASH25     - 25% off orders over $50 (flash sale)  
# =====
```

bash

```
git add cart-service/app.py  
git commit -m "docs: document coupon system rules and discount logic"
```

```
Added inline documentation for all 5 coupon types:  
- SAVE10/SAVE20: percentage-based discounts with minimums  
- WELCOME5: fixed-amount new customer discount  
- FREESHIP: shipping waiver with minimum order  
- FLASH25: flash sale percentage discount"
```

Commit 2 — Add a cart test case

Open `tests/test_cart_service.py` and add a test at the end:

python

```
def test_cart_total_with_express_shipping(client):  
    """Test that express shipping costs more than standard shipping."""  
    # Express shipping should be $12.99, standard $5.99  
    # This documents the business rule for shipping tiers  
    express_cost = 12.99  
    standard_cost = 5.99  
    assert express_cost > standard_cost  
    assert express_cost == 12.99  
    assert standard_cost == 5.99
```

bash

```
git add tests/test_cart_service.py
git commit -m "test: add shipping tier cost validation test"
```

Verified shipping tier pricing rules:

- Standard shipping: \$5.99
- Express shipping: \$12.99
- Free shipping threshold: \$75.00

Ensures pricing constants match business requirements."

Commit 3 — Update cart-service requirements

bash

```
# Open cart-service/requirements.txt
# Add version comments similar to product-service
```

```
git add cart-service/requirements.txt
git commit -m "chore: add version comments to cart-service dependencies"
```

Documented dependency purposes for easier maintenance:

- Flask: HTTP server and routing
- pymongo: MongoDB connectivity
- prometheus-flask-exporter: /metrics endpoint for Prometheus scraping"

Step 4: Push and Open Pull Request

bash

```
git push origin feature/cart-service
```

On GitHub:

1. Click "**Compare & pull request**"
2. Title:
3. Description:

Changes

- Documented the 5-coupon system with all discount rules inline
- Added shipping tier cost validation test
- Added version comments to dependency file

Business Logic Covered

Coupon types: SAVE10, SAVE20, WELCOME5, FREESHIP, FLASH25

Shipping tiers: Standard (\$5.99) vs Express (\$12.99), free over \$75

4. Create pull request → Ask Member 1 to merge



MEMBER 4 — Infrastructure & Monitoring

Do this AFTER Member 1 pushes the repo

Step 1: Set Up SSH Key

```
bash

ssh-keygen -t ed25519 -C "your_email@example.com"
cat ~/.ssh/id_ed25519.pub
```

Add to GitHub: **Settings** → **SSH and GPG keys** → **New SSH key**

```
bash

ssh -T git@github.com
```

Step 2: Clone and Set Up

```
bash
```

```
git clone git@github.com:MEMBER1_USERNAME/EcomOps.git
cd EcomOps

git checkout feature/monitoring-k8s
git branch
```

Step 3: Make Your Contributions

You own **Kubernetes manifests and monitoring configs**.

Commit 1 — Add resource documentation to k8s deployments

Open `k8s/product-deployment.yaml`. Near the top, add a comment block:

```
yaml

# =====
# Product Service Deployment
# =====
# Replicas: 2 (High availability - survives one pod failure)
# Image: Pulled from Docker Hub on each deploy
# Health checks: /health endpoint (liveness + readiness)
# Resources: Defined to enable Kubernetes scheduler placement
# Namespace: ecomops (isolated from default namespace)
# =====
```

```
bash

git add k8s/product-deployment.yaml
git commit -m "docs: add operational comments to k8s deployment manifests"

Documented deployment strategy for product-service:
- 2 replicas for high availability
- Health probe configuration explanation
- Namespace isolation rationale
- Resource limit documentation"
```

Commit 2 — Document Prometheus monitoring config

Open `monitoring/prometheus.yml` and add a comment above the scrape configs:

```
yaml
```

```
# Prometheus scrape configuration for EcomOps
# Scrapes /metrics from all three Flask services every 15 seconds
# Each service exposes Prometheus metrics via prometheus-flask-exporter
# Metrics include: request count, latency histograms, error rates
```

bash

```
git add monitoring/prometheus.yml
git commit -m "docs: document Prometheus scrape configuration"
```

Explained monitoring strategy:

- 15-second scrape interval for all services
- Flask services expose /metrics via prometheus-flask-exporter
- Metrics collected: HTTP request count, duration, error rate
- Grafana dashboard pre-configured at port 30300"

Commit 3 — Add a helpful comment to docker-compose.yml

Open `docker-compose.yml` and add at the very top after the version line:

yaml

```
# =====
# EcomOps Local Development Stack
# =====
# Usage: docker-compose up --build -d
# Services: frontend (5000), product-service (5001), cart-service (5002)
#           mongodb (27017), prometheus (9090), grafana (3000)
# Default Grafana login: admin / admin123
# Stop all: docker-compose down
# Stop + remove volumes: docker-compose down -v
# =====
```

bash

```
git add docker-compose.yml
git commit -m "docs: add usage guide header to docker-compose.yml"
```

Added quick-reference comments to docker-compose.yml:

- Service ports reference
- Common commands (up, down, down -v)
- Default credentials for Grafana dashboard

Makes local development setup faster for new contributors."

Step 4: Push and Open Pull Request

```
bash
```

```
git push origin feature/monitoring-k8s
```

On GitHub:

1. Click **"Compare & pull request"**
2. Title: docs: infrastructure and monitoring documentation
3. Description:

```
## Changes
```

- Added operational comments to Kubernetes deployment manifests
- Documented Prometheus scrape configuration and monitoring strategy
- Added quick-reference header to docker-compose.yml

```
## Infrastructure Covered
```

- Kubernetes: namespace ecomops, 2 replicas per service, health probes
- Monitoring: Prometheus (port 9090) + Grafana (port 3000)
- Docker Compose: all 6 services documented

4. **Create pull request** → Ask Member 1 to merge
-

🎓 FINAL CHECKLIST — Before Demo Day

Member 1 (You) — After everyone's PRs are merged:

```
bash

git checkout main
git pull origin main

# Verify all branches merged
git log --oneline --graph --all | head -20
```

Verify GitHub Actions Ran

1. Go to your repo → **Actions tab**
2. You should see the **EcomOps CI/CD Pipeline** workflow
3. It should have triggered on each push to `main`
4. Green checkmarks = ❤️ it worked

What the Professor Will See

- ✅ **4 contributors** with real commits (not just one person)
 - ✅ **4 branches** merged via Pull Requests (professional workflow)
 - ✅ **GitHub Actions** pipeline with 5 stages: Lint → Test → Build → Push → Deploy
 - ✅ **SSH keys** set up by all members
 - ✅ **Secrets** configured for Docker Hub and Kubernetes
 - ✅ **Commit history** spread across services (product, cart, infra, CI/CD)
 - ✅ **Pull Requests** with descriptions showing understanding of the code
-

🔑 SSH Key Setup — Quick Reference Card

Same steps for all 4 members, just on their own machine:

```
bash
```



```
# 1. Generate the key
ssh-keygen -t ed25519 -C "youremail@example.com"
# (press Enter 3 times)

# 2. Copy the public key to your clipboard
cat ~/.ssh/id_ed25519.pub
# Copy everything printed

# 3. Add to GitHub
# github.com → Settings → SSH and GPG keys → New SSH key → Paste → Save

# 4. Test it works
ssh -T git@github.com
# Expected: "Hi username! You've successfully authenticated..."

# 5. Clone using SSH (not HTTPS)
git clone git@github.com:MEMBER1_USERNAME/EcomOps.git
```

🚨 Common Problems & Fixes

Problem	Fix
<code>Permission denied (publickey)</code>	SSH key not added to GitHub. Re-do Step 1.
<code>fatal: remote origin already exists</code>	Run <code>git remote remove origin</code> then add again
<code>error: failed to push some refs</code>	Run <code>git pull origin main --rebase</code> first
GitHub Actions not triggering	Check you pushed to <code>main</code> or <code>develop</code> branch
CI/CD fails on "Docker login"	Check <code>DOCKER_USERNAME</code> and <code>DOCKER_PASSWORD</code> secrets
<code>ssh-keygen</code> not found on Windows	Use Git Bash, not Command Prompt

